

Slovenská technická univerzita v Bratislave

Fakulta elektrotechniky a informatiky, Ilkovičova 3, 812 19 Bratislava

Zadanie č.2

Kalibrácia kamery

Dokumentácia z predmetu: **Počítačové videnie a spracovanie obrazu**

Vypracovali: Maslen Jakub, Kormaníková Dominika
Rok: 2025–2026, 1. ročník Ing.

Obsah

1	Kalibrácia kamery	3
1.1	Získanie kalibračných snímok	3
1.2	Výpočet kalibračných parametrov	3
1.3	Kalibrácia v reálnom čase	4
2	Detekcia geometrických tvarov	5
2.1	Predspracovanie obrazu	5
2.2	Detekcia kružníc	5
2.3	Detekcia trojuholníkov, štvorcov a obdĺžnikov	5
3	Farebný filter	8

1 Kalibrácia kamery

Cieľom tejto úlohy bola kalibrácia kamery za účelom odstránenia skreslenia obrazu. Kalibráciou sme získali vnútorné parametre kamery a distorzné koeficienty skreslenia, ktoré sme následne použili na kalibráciu kamery a na demonštrovanie neskresleného obrazu v reálnom čase.

Proces kalibrácie sme rozdelili do troch krokov:

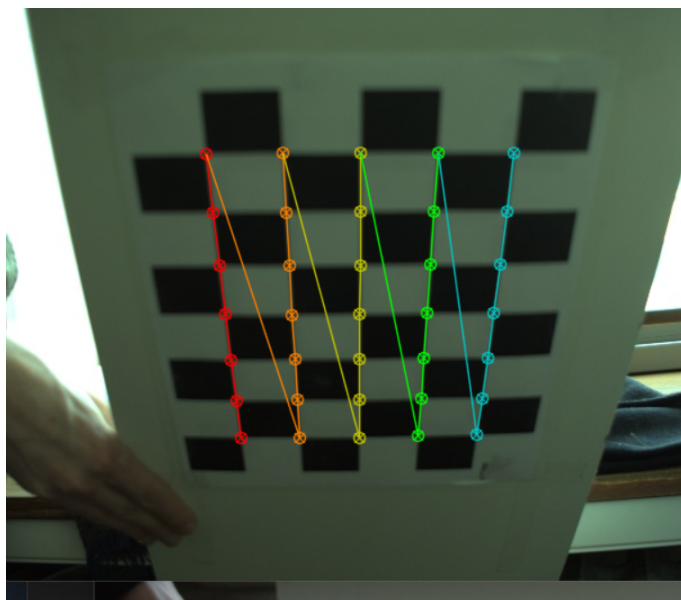
- získanie kalibračných snímok,
- výpočet kalibračných parametrov,
- aplikácia kalibrácie v reálnom čase.

1.1 Získanie kalibračných snímok

Kalibráciu kamery sme realizovali pomocou šachovnicového kalibračného vzorca podľa oficiálneho postupu v dokumentácii OpenCV pre kalibráciu kamery.

Najskôr sme pomocou kamery nasníмали väčší počet obrázkov, u nás to bolo 50 snímok, na ktorých sa šachovnica nachádzala v rôznych polohách, orientáciách a vzdialenostiach od kamery. Cieľom bolo zachytiť kalibračný vzor z čo najväčšieho počtu uhlov, aby sme boli schopní, čo najpresnejšie vypočítať parametre kamery.

Z celkového počtu získaných snímok sme následne vybrali 14 obrázkov, ktoré obsahovali šachovnicu z rôznych uhlov a zároveň mali dobre viditeľné všetky rohy šachovnice. Tieto snímky sme použili ako vstupné dáta pre výpočet kalibračných parametrov.



Obr. 1: Príklad kalibračnej snímky šachovnicového vzoru

1.2 Výpočet kalibračných parametrov

Na základe vybraných snímok sme pomocou knižnice OpenCV detegovali rohy šachovnice pomocou funkcie `findChessboardCorners`. Tieto body predstavujú pro-

jekciu známych bodov kalibračného vzoru do obrazovej roviny. Použili sme kalibračný vzor s rozmermi 7×5 vnútorných rohov. Po úspešnej detekcii sme pozície rohov spresnili pomocou funkcie `cornerSubPix`, ktorá umožňuje presnejšiu lokalizáciu bodov.

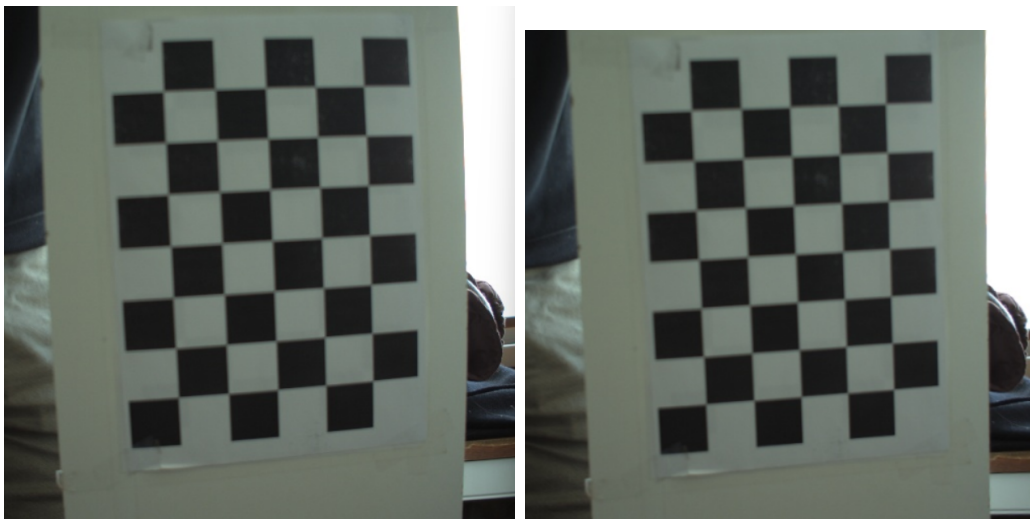
Na základe získaných bodov sme pomocou funkcie `calibrateCamera` vypočítali vnútorné parametre kamery a koeficienty skreslenia objektívu. Výsledkom kalibrácie bola tzv. matica kamery (*camera matrix*), ktorá obsahuje základné parametre optiky kamery. Matica mala nasledujúci tvar:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Z tejto matice sme získali hodnoty:

- $f_x = 941.402$ a $f_y = 940.708$ – ohnisková vzdialenosť v pixeloch,
- $c_x = 313.361$ a $c_y = 254.709$ – súradnice optického stredu kamery.

Pre overenie správnosti kalibrácie sme aplikovali funkciu `undistort`, ktorá odstránila skreslenie spôsobené objektívom kamery.



Obr. 2: Porovnanie pôvodného a kalibrovaného obrazu

Vypočítané kalibračné parametre sme následne uložili do súborov `calibration_data.npz` a `calibration.pkl`, aby sme ich mohli použiť v ďalšej časti úlohy.

1.3 Kalibrácia v reálnom čase

V poslednom kroku sme aplikovali získané kalibračné parametre na obraz z kamery v reálnom čase. Najskôr sme načítali uložené parametre zo súboru `calibration.pkl`. Následne sme opäť inicializovali kameru a nastavili rovnaké parametre snímania ako v prvom programe.

Obraz získaný z kamery sme opäť upravili pomocou funkcie `resize` na rozlíšenie 616×514 pixelov, aby zodpovedal rozlíšeniu použitému počas kalibrácie. Každý získaný frame sme následne spracovali pomocou funkcie `undistort`, ktorá odstránila skreslenie objektívu.

2 Detekcia geometrických tvarov

V druhej úlohe sme mali detegovať základné geometrické tvary v reálnom čase na obraze zo snímacej kamery a graficky ich označiť v obraze.

Obraz sme získavali zo snímacej kamery pomocou knižnice `OpenCV`. Na začiatku sme načítali kalibračné parametre kamery, ktoré sme získali v predchádzajúcej úlohe kalibrácie. Tým sme zabezpečili, že detekcia tvarov prebiehala na geometricky korigovanom obraze.

2.1 Predspracovanie obrazu

Po získaní snímky z kamery sme najskôr upravili jej rozlíšenie na hodnotu 616×514 pixelov, aby bolo spracovanie obrazu konzistentné s predchádzajúcou kalibráciou.

Následne sme obraz previedli do odtieňov sivej pomocou funkcie `cvtColor`. Tento krok zjednodušuje spracovanie obrazu, pretože detekcia hrán a kontúr nevyžaduje farebnú informáciu. Na zníženie šumu sme použili Gaussovo rozmazanie (`GaussianBlur`).

2.2 Detekcia kružníc

Kružnice sme detegovali pomocou Houghovej transformácie, ktorá je implementovaná vo funkcii `HoughCircles`. Táto metóda dokáže identifikovať kruhové objekty na základe detegovaných hrán v obraze.

Funkcia analyzuje gradienty v obraze a vyhľadáva body, ktoré môžu patriť kružniciam s určitým polomerom. Ak bola kružnica detegovaná, program vykreslil jej obvod a zároveň označil jej stred malým bodom.

2.3 Detekcia trojuholníkov, štvorcov a obdĺžnikov

Pre detekciu ostatných geometrických tvarov sme najskôr vykonali prahovanie obrazu pomocou funkcie `threshold`. Tento krok vytvoril binárny obraz, ktorý zvýraznil objekty na pozadí. Následne sme použili algoritmus Cannyho detektora hrán (`Canny`), ktorý umožňuje presne identifikovať hrany objektov v obraze.

Pre každú nájdenú kontúru sme najskôr vypočítali jej plochu pomocou funkcie `contourArea`. Kontúry s veľmi malou plochou sme ignorovali. Následne sme kontúru aproximovali pomocou funkcie `approxPolyDP`, ktorá zjednodušuje tvar kontúry na polygonálny tvar s menším počtom vrcholov. Pomocou počtu vrcholov sme dokázali

určiť typ geometrického tvaru: 3 vrcholy – trojuholník a 4 vrcholy – štvorec alebo obdĺžnik. V prípade štyroch vrcholov sme rozlíšili štvorec a obdĺžnik pomocou pomeru strán (`boundingRect`). Ak bol pomer šírky a výšky približne rovný jednej, objekt sme klasifikovali ako štvorec, inak ako obdĺžnik.

Pre každý detegovaný objekt sme vypočítali jeho stred pomocou obrazových momentov (`moments`). Zo získaných momentov sme vypočítali súradnice ťažiska objektu podľa vzťahov:

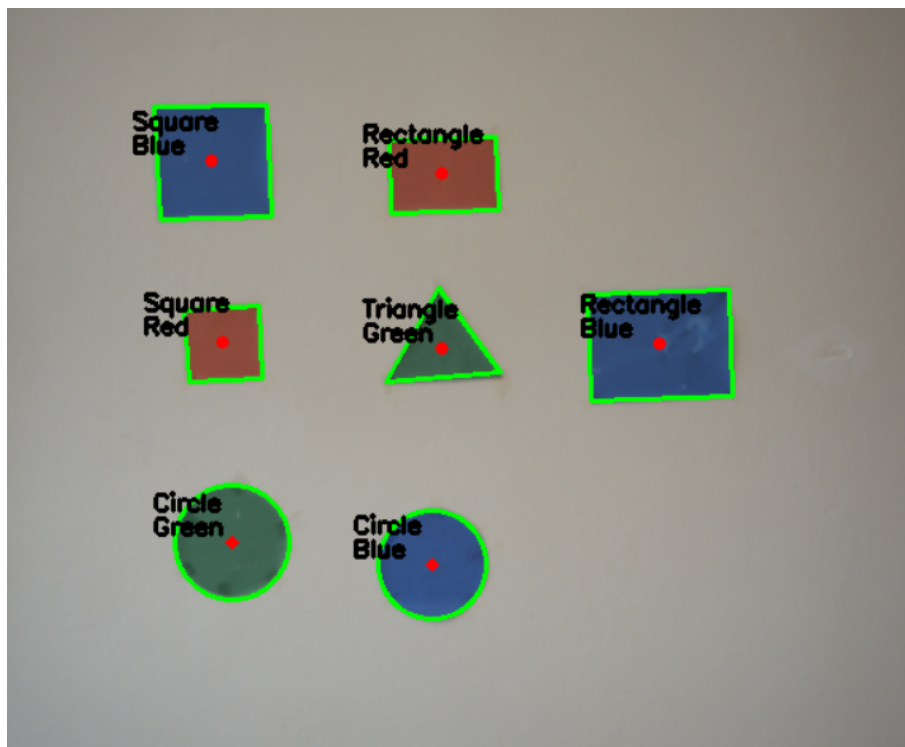
$$c_x = \frac{m_{10}}{m_{00}}, \quad c_y = \frac{m_{01}}{m_{00}}$$

Tieto súradnice predstavujú geometrický stred objektu. Stred sme následne zobrazili priamo v obraze pomocou malej bodky.

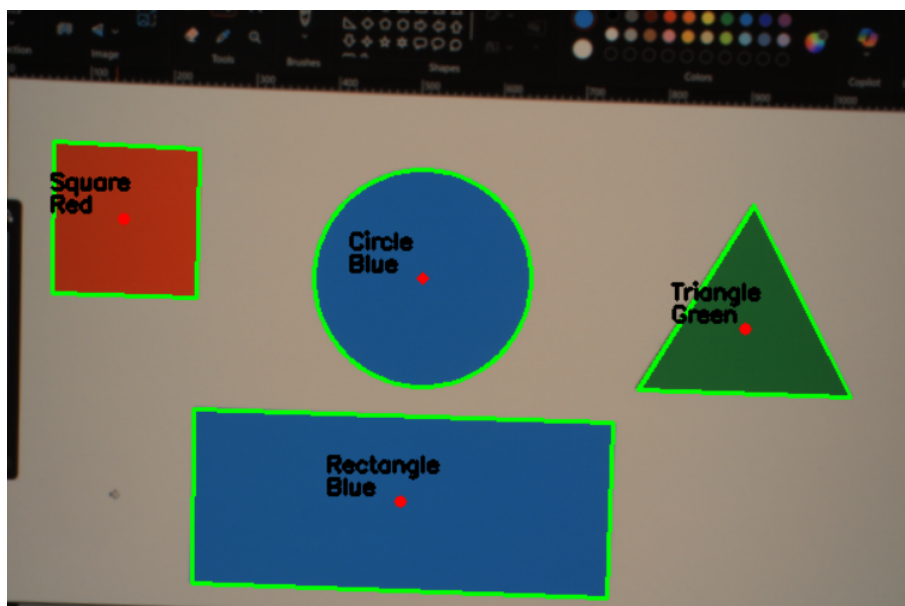
Aplikovali sme aj rozpoznávanie farby podľa zadania. Najskôr sme obraz previedli z farebného priestoru BGR do farebného priestoru HSV pomocou funkcie `cvtColor`. HSV farebný priestor je vhodnejší na detekciu farieb, pretože oddeľuje informáciu o odtieni farby od jej jasnosti.

Pre každý detegovaný objekt sme vytvorili masku, ktorá obsahovala iba pixely patriace danému objektu. Následne sme vypočítali priemernú hodnotu farby v tejto oblasti pomocou funkcie `mean`.

Na základe hodnoty odtieňa (Hue) sme objekt klasifikovali ako jednu zo základných farieb: červená, žltá, zelená a modrá. Výsledkom programu bola detekcia geometrických tvarov v reálnom čase. Detegovaný objekt bol zároveň graficky označený v obraze pomocou obrysu kontúry a textového popisu.



Obr. 3: Výsledok detekcie geometrických tvarov a ich farieb



Obr. 4: Výsledok detekcie geometrických tvarov a ich farieb

3 Farebný filter

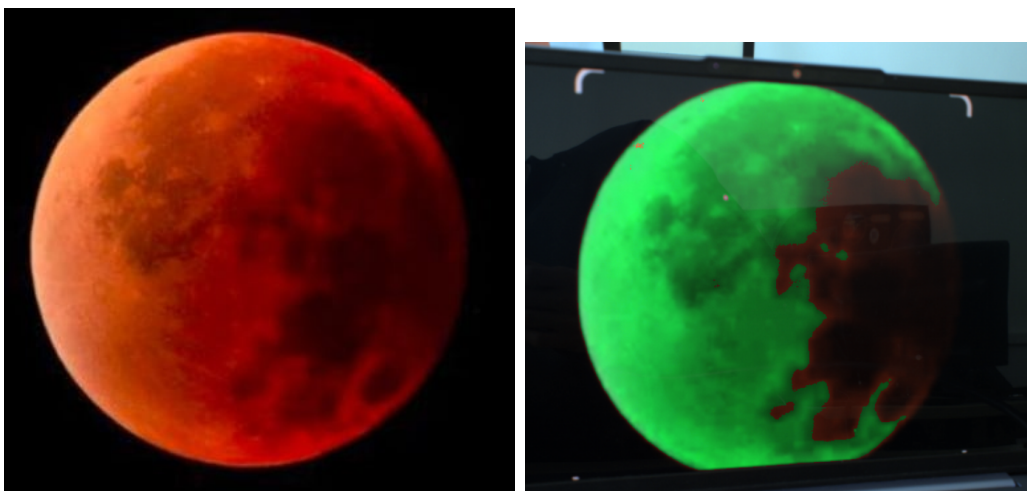
V tretej úlohe sme implementovali farebný filter, ktorý umožnil vybrať konkrétnu farbu v obraze a nahradiť ju inou zvolenou farbou. Program pracoval v reálnom čase s obrazom zo snímacej kamery.

N začiatku sme rovnako ako v druhej úlohe aplikovali kalibračné parametre na odstránenie skreslenia kamery. Potom sme obraz z kamery previedli z farebného priestoru BGR do farebného priestoru HSV pomocou funkcie `cvtColor`. Farebný priestor HSV je vhodnejší na detekciu farieb, pretože oddeľuje informáciu o odtieni farby od jasnosti a sýtosti. Teda je vhodný ak chceme prevádzať tmavšie a svetlejšie odtiene.

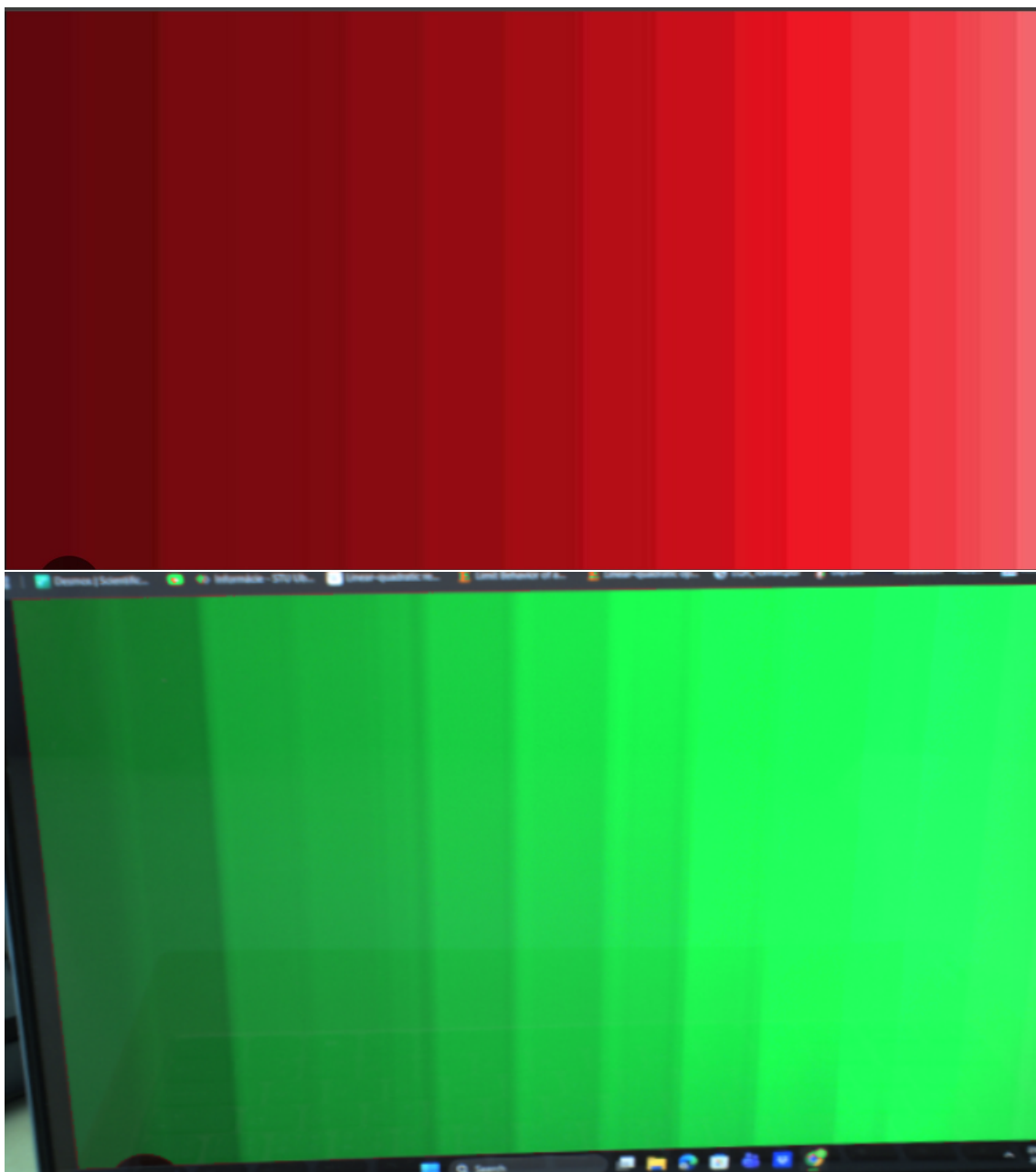
Pre detekciu konkrétnej farby sme definovali rozsah hodnôt odtieňa v HSV priestore. Pomocou funkcie `inRange` sme vytvorili binárnu masku, ktorá obsahovala pixely patriace do zvoleného farebného rozsahu.

Táto maska označovala všetky pixely, ktoré mali požadovanú farbu. V poslednom kroku sme všetky pixely označené v maske nahradili novou zvolenou farbou. V našom prípade sme napríklad všetky červené objekty zobrazili ako zelené.

Takýmto spôsobom bolo možné dynamicky meniť farbu vybraných objektov priamo v obraze z kamery.



Obr. 5: Výsledok aplikácie farebného filtra



Obr. 6: Výsledok aplikácie farebného filtra